Anomaly Data Evolution — Raw UART to ML Result, End to End

How a row of raw UART text becomes an ML result, for both fault classes, following the exact pipeline the campaign runs. Numbers below are from the current dataset (`datasets/*.csv`).

The one idea to hold onto the whole way:

> **Memory leak → we PREDICT a number** (seconds to crash). **Deadline miss → we DETECT a state** (is it wrong right now?). **Both classes → we PERSIST a detector** (the int8 model that ships on the M7).

Legend — where every column comes from

Every column has exactly one of three origins. Know which, and the whole pipeline reads cleanly.

COLUMN	ORIGIN	PRODUCED BY	HOW
`heap_free`, `heap_used`	**Measured**	K1 firmware	Zephyr heap stats, reported each tick
`loop_latency`	**Measured**	K1 firmware	stopwatch: `work_ms = k_uptime_get() − t0` around the loop body
`heap_free_slope`, `loop_latency_slope`	**Arithmetic**	`dataset.py`	rolling first-difference ÷ Δt (a numerical derivative)
`true_ttf`	**Arithmetic**	`run_campaign.py`	`heap_free ÷ leak_rate` — **memory leak only**
`faulty`	**Label**	`run_campaign.py`	`True` for every row at/after the injection instant

The three formulas

1 · Measured latency is a stopwatch, not a computation. A healthy loop does almost nothing → `work_ms ≈ 0–1`. The deadline fault calls `k_busy_wait()` for a ramping number of microseconds, so the same stopwatch reads higher and higher.

2 · Slope (both `_slope` columns) — the rate a signal is moving, in units/second:

```
slope = series.diff().rolling(window).mean() / dt_s        # dt_s = 0.1 s
```

Example: `loop_latency` steps `0 → 6` in one 0.1 s tick → slope ≈ `6 / 0.1 = +60/s`. A single value says little; its *slope* is what actually flags a fault.

3 · True time-to-failure — the analytical ground truth, memory leak only:

```
leak_rate = leak_bytes_per_tick × tick_hz                  # e.g. 128 × 10 = 1280 B/s
true_ttf  = heap_free / leak_rate                          # seconds until free heap hits 0
```

Worked from real rows (rate 128): `7976 / 1280 = 6.231 s`, `88 / 1280 = 0.069 s`. It is *exact* because you **set** the leak rate and the firmware **reports** the bytes left — so the predictor's guess can be graded against a known-correct answer. Deadline-miss has no such analytical crash point, so its `true_ttf` is always blank.

One more transform the detectors apply — standardization

Before the detectors see the features, each is z-scored so no single column dominates by scale:

```
z = (x − μ) / σ            # μ, σ computed on TRAIN-NORMAL rows only (never the test set)
```

Fitting μ/σ on train-normal only is a hard rule (no test leakage). `FEATURES` order is fixed everywhere downstream: `[heap_free, heap_free_slope, loop_latency, loop_latency_slope]`.

Stage 0 — Raw UART text (identical format, different content)

Firmware prints `TELEM,uptime_ms,node,signal,seq,value`. Signals: `1=heap_free`, `2=heap_used`, `5=loop_latency`.

Memory leak — `heap_free` falls, latency steady:

```
TELEM,200,1,1,0,8112      heap_free = 8112  (full)
TELEM,111343,1,1,55,7976  heap_free dropping
TELEM,120680,1,1,99,88    heap_free almost gone
```

Deadline miss — `heap_free` steady, latency climbs:

```
TELEM,200,1,5,0,0         loop_latency = 0  (fine)
TELEM,59040,1,5,50,6      loop_latency rising
TELEM,61221,1,5,99,81     loop_latency huge
```

Same text shape — the fault lives in a *different signal*.

Stage 1 — `run_campaign.py` labels it → CSV (long format)

Parses each TELEM line, marks rows `faulty` after injection, and — **only for the prediction class** — fills `true_ttf` (Legend formula 3).

Memory leak (`ttf_sig = heap_free` → filled):

TIMESTAMP	SIGNAL	VALUE	LABEL	TRUE_TTF
200	heap_free	8112	normal	
111343	heap_free	7976	faulty	6.231
120680	heap_free	88	faulty	0.069

Deadline miss (`ttf_sig = None` → always blank):

TIMESTAMP	SIGNAL	VALUE	LABEL	TRUE_TTF
200	loop_latency	0	normal	
59040	loop_latency	6	faulty	
61221	loop_latency	81	faulty	

The fork is born here: the leak gets a target column; deadline-miss never does.

Stage 2 — `dataset.py · pivot_wide` (long → wide, one row per tick)

Signals become columns; `faulty` and `true_ttf` merge back onto each 100 ms tick.

EPISODE	TIMESTAMP	HEAP_FREE	HEAP_USED	LOOP_LATENCY	FAULTY	TRUE_TTF
0	200	8112	0	0	False	NaN
0	111300	7976	136	0	True	6.231

Stage 3 — `dataset.py · add_features` (add slopes)

Adds `heap_free_slope` and `loop_latency_slope` (Legend formula 2). Up to here **the code is identical for both classes** — same functions, same frame.

TIMESTAMP	HEAP_FREE	HEAP_FREE_SLOPE	LOOP_LATENCY	LOOP_LATENCY_SLOPE	FAULTY	TRUE_TTF
200	8112	0.0	0	0.0	False	NaN
111300	7976	−1360.0	0	0.0	True	6.231

Now the pipeline forks into **three destinations**.

Fork A — `baseline.py` · DETECT (both classes)

No ramp filter — keep every row and z-score it (detection needs *normal* rows to learn "normal"). Fit two rival detectors on train-normal, score how weird each test row is.

```
Xtr_normal = z-scored features, NORMAL rows only     (train)
Xte, yte   = z-scored features + labels, both classes (test; y: 0=normal, 1=faulty)
```

DETECTOR	MEMORY_LEAK	DEADLINE_MISS	READING
Isolation Forest	**0.513**	**0.995**	classical; blind to gradual drift, great on step-changes
Denoising AE	**1.000**	**1.000**	learns normal's shape; catches both

(ROC-AUC. FP/hour ≈ 0 at the chosen threshold — see Fork C.) **Output = a verdict:** "abnormal now?" Keeping both is the result: they fail on opposite data shapes, so the contrast is the finding.

Fork B — `predictor.py` · PREDICT (memory leak only)

Ramp filter keeps only the draining rows (`faulty & heap_free_slope < 0 & true_ttf > 0.1`), then splits into `X = [heap_free, heap_free_slope]` , `y = true_ttf` . Two predictors race, scored in seconds against the analytical truth:

PREDICTOR	WHAT IT IS	MAE
analytic	`heap_free ÷ −slope` (zero-parameter physics)	**2.53 s**
linreg	learned `ttf ≈ a·heap_free + b`	**3.04 s**

Honest result: with **mixed leak rates** the physics baseline *beats* the learned model — one linear fit can't serve 64/128/256 B/tick at once, but the division handles each rate exactly. **Output = a time:** "K1 crashes in X seconds." Deadline-miss has no `true_ttf` , so it never enters this fork.

Fork C — `train_ae.py` · PERSIST the detector (both classes) → on-chip

`baseline.py` trains-and-discards; `train_ae.py` trains the denoising AE on normal and **saves the artifact Sprint 2 quantizes**: `ae.pt` (weights + the exact μ/σ scaler + threshold), `calib.npy` (frozen int8 calibration set), `ae_manifest.json` (provenance).

Threshold policy replaces the old `q=0.99` (which clipped 1% of normal → 223 FP/hour) with a line just past the worst training-normal score:

```
threshold = max(train-normal score) × 1.10
```

CLASS	AE AUC	THRESHOLD	FP/HOUR
memory_leak	1.000	4.63	**0.00**
deadline_miss	1.000	0.875	**0.00**

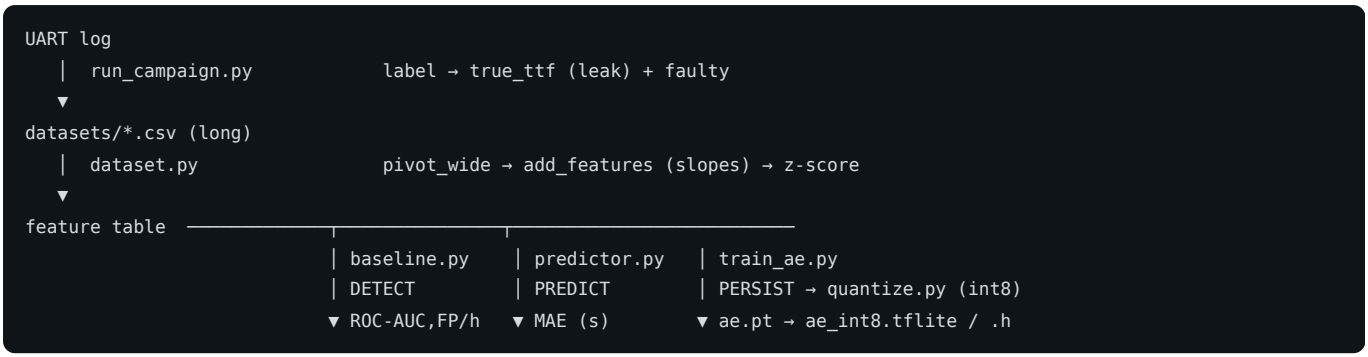
Then `quantize.py` converts the saved float AE → **int8 LiteRT** for the M7, with a parity gate (float vs int8 AUC drop must stay < 2 pts; measured drop **0.000**). **Output = a deployable int8 model** + a C header the K3 firmware embeds.

The whole thing in one glance

STAGE	MEMORY LEAK	DEADLINE MISS
raw signal that moves	`heap_free` down	`loop_latency` up
`true_ttf` column	**filled** (÷ leak_rate)	**empty**
shared: long→wide→slopes	same code	same code
Fork A · DETECT (`baseline.py`)	IsoForest 0.513 / AE 1.000	IsoForest 0.995 / AE 1.000
Fork B · PREDICT (`predictor.py`)	analytic 2.53 s / linreg 3.04 s	— (no ground truth)
Fork C · PERSIST (`train_ae.py`)	AE saved, FP/h 0.00 → int8	AE saved, FP/h 0.00
answers	"crash in X s" **and** "abnormal now?"	"abnormal now?"

The mental hook: both classes walk *identical* rails from raw text to the slope-feature table. Then the `true_ttf` column — present for the leak, absent for timing — decides which forks each class can take: everything **detects** and gets **persisted**; only the leak, with its analytical countdown, also **predicts**.

Script map


UART log
run_campaign.py
label → true_ttf (leak) + faulty
datasets/*.csv (long)
dataset.py
pivot_wide → add_features (slopes) → z-score
feature table
baseline.py
DETECT
ROC-AUC,FP/h
predictor.py
PREDICT
MAE (s)
train_ae.py
PERSIST → quantize.py (int8)
ae.pt → ae_int8.tflite / .h